

Apache Kafka Spring Boot Lab Guide

Build a Spring Boot REST API that sends and reads the CLI lab's orders, then watch partitions, offsets, consumer groups and broker failures from Java. Every output was captured from the app running against the lab's three-broker Kafka 4.3.1 cluster.

SPRING BOOT 4

KAFKATEMPLATE

@KAFKALISTENER

REST API

OFFSETS

CONSUMER GROUPS

ADMIN API

0 Overview and Setup

1 Build and Run the App

2 The Code, File by File

3 Send Messages over REST

4 Get Messages over REST

5 Failures and Scaling

6 API Reference and Troubleshooting

0

1

2

3

4

5

6

7

8

9

10

11

Lab 0 · Overview and Setup

Goal: see what the app does, check that you have everything it needs, and find your way around the project.

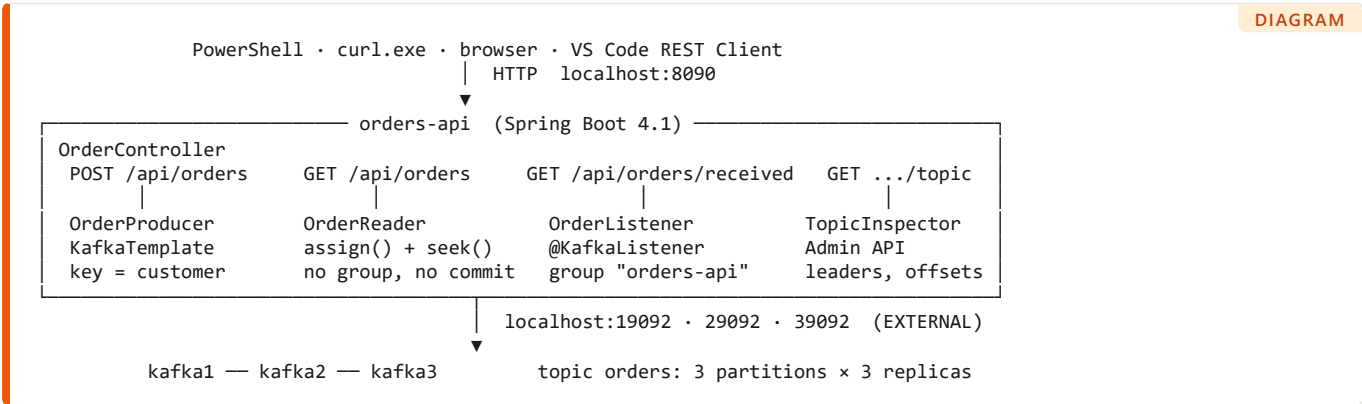
What you will build

`orders-api` is a small Spring Boot 4 application that connects to the three-broker cluster from the **Kafka CLI Lab**. It uses the same `orders` topic you created in Lab 2 and filled in Lab 5. The key of each message is the customer, and the value is the order as JSON:

```
alice:{"order":2001,"item":"keyboard","amount":45}
```

OUTPUT

The app does with Java code what you did with the command-line tools. It sends orders through a REST endpoint and receives them with a consumer group. It can read the topic from any offset and describe the topic's partitions.



Method	Path	What it does	Kafka API underneath
POST	/api/orders	Sends one order. Returns the partition and offset it was stored at	<code>KafkaTemplate.send()</code> - a producer
GET	/api/orders/received	The last 50 messages the listener received, newest first	<code>@KafkaListener</code> - consumer group <code>orders-api</code>
GET	/api/orders	Reads the topic from any offset, optionally one partition or one customer	a consumer with <code>assign()</code> and <code>seek()</code>
GET	/api/orders/topic	Leader, replicas, ISR, first and next offset of every partition	<code>Admin.describeTopics()</code> , <code>listOffsets()</code>

What you need

What	Why	Check
The CLI lab cluster, running	The app connects to it	<code>docker compose ps</code> in <code>kafka_cli_lab</code> shows three brokers <code>(healthy)</code>
JDK 21 or newer	Spring Boot 4 needs Java 17+. The project targets 21	<code>java -version</code>
Internet access for the first build	The Maven wrapper downloads Maven 3.9.16 and the libraries	-
<code>curl.exe</code>	Ships with Windows 10 and 11	<code>curl.exe --version</code>

You do **not** need to install Maven: `mvnw.cmd` downloads the right version the first time it runs. An IDE is optional. IntelliJ IDEA, or VS Code with the *Extension Pack for Java*, opens the `orders-api` folder as a Maven project.

TWO SHELLS IN THIS BOOK

Blocks labelled **POWERSHELL** run on Windows. Run them in the `orders-api` folder unless the text says otherwise. Blocks labelled **KAFKA1 SHELL** run inside the broker container: open one with `docker exec -it kafka1 bash`, as in the CLI lab.

VERSIONS USED IN THIS BOOK

Spring Boot 4.1.1, Spring for Apache Kafka 4.1.1, Kafka clients 4.2.1, Java 21 and Maven 3.9.16. The brokers are Apache Kafka 4.3.1 from the CLI lab. A 4.2 client talks to a 4.3 broker without any problem: clients and brokers negotiate the API versions they share.

The files

DIAGRAM

```
kafka_spring_boot_lab/
├── README.md
├── Kafka_Spring_Boot_Lab_Guide.pdf
├── send-orders.ps1
├── guide/
├── build_pdf.py
├── orders-api/
│   ├── pom.xml
│   ├── mvnw, mvnw.cmd, .mvn/
│   ├── requests.http
│   ├── samples/order.json
│   └── src/main/
│       ├── resources/application.yml
│       └── java/com/training/kafka/orders/
│           ├── OrdersApiApplication.java
│           ├── KafkaTopicConfig.java
│           ├── Order.java
│           ├── OrderRequest.java
│           ├── SendReceipt.java
│           ├── OrderMessage.java
│           ├── TopicInfo.java
│           ├── OrderProducer.java
│           ├── OrderListener.java
│           ├── OrderReader.java
│           ├── TopicInspector.java
│           ├── OrderController.java
│           ├── ApiExceptionHandler.java
│           └── KafkaUnavailableException.java
```

this book
posts every line of orders.txt to the API
the chapters of this book (Markdown)
builds the PDF
the Spring Boot project
Maven wrapper - no Maven install needed
ready-made requests for VS Code / IntelliJ
a request body for curl.exe
connection and client settings
main()
creates "orders" if it is missing
the message value
the POST body, validated
the POST response
a message read back
the /topic response
sends - KafkaTemplate
receives - @KafkaListener
reads from any offset
describes - Admin API
the REST endpoints
errors as problem JSON

From the CLI lab to the app

In the CLI lab you used

```
kafka-topics.sh --create --topic orders --partitions 3 --
replication-factor 3
```

```
kafka-console-producer.sh --reader-property parse.key=true
```

```
lab/config/producer.properties: acks=all, idempotence, linger.ms,
lz4
```

```
kafka-console-consumer.sh --group ...
```

```
kafka-console-consumer.sh --partition 0 --offset 2 --max-messages
3
```

In this app

The `NewTopic` bean in `KafkaTopicConfig`

```
kafkaTemplate.send(topic, customer, json)
```

```
spring.kafka.producer.* in application.yml
```

```
@KafkaListener(groupId = "orders-api")
```

```
GET /api/orders?partition=0&offset=2&limit=3
```

In the CLI lab you used	In this app
<code>kafka-consumer-groups.sh --describe --group ...</code>	Works on the app's group <code>orders-api</code> too - try it in Lab 4
<code>kafka-topics.sh --describe --topic orders</code>	<code>GET /api/orders/topic</code>

Lab 1 · Build and Run the App

Goal: start the cluster, build `orders-api` with the Maven wrapper, run it, and check that it found the `orders` topic, or created it.

Exercise 1.1 · Start the cluster

Task: start the three brokers from the CLI lab and wait until all three are healthy.

✓ SOLUTION

POWERSHELL

```
cd D:\trainings\Apache_Kafka_Training\labs\kafka_cli_lab
docker compose up -d
docker compose ps
```

OUTPUT

NAME	IMAGE	STATUS	PORTS
kafka1	apache/kafka:4.3.1	Up 2 hours (healthy)	0.0.0.0:19092->19092/tcp
kafka2	apache/kafka:4.3.1	Up 5 minutes (healthy)	0.0.0.0:29092->29092/tcp
kafka3	apache/kafka:4.3.1	Up 2 hours (healthy)	0.0.0.0:39092->39092/tcp

(Columns trimmed.) The `PORTS` column matters for the app. Each broker publishes its **EXTERNAL** listener on the Windows host: `localhost:19092`, `29092` and `39092`. Code running on Windows has to use those addresses. Inside Docker, the brokers call each other `kafka1:9092` and so on, and those names mean nothing to Windows.

Exercise 1.2 · Build

Task: build the application jar with the Maven wrapper.

✓ SOLUTION

POWERSHELL

```
cd D:\trainings\Apache_Kafka_Training\labs\kafka_spring_boot_lab\orders-api
.\mvnw.cmd -DskipTests package
```

The first run downloads Maven itself and then all the libraries into `%USERPROFILE%\m2`, which takes a minute or two. Later builds take seconds:

OUTPUT

```
[INFO] --- jar:3.5.1:jar (default-jar) @ orders-api ---
[INFO]
[INFO] --- spring-boot:4.1.1:repackage (repackage) @ orders-api ---
[INFO] Replacing main artifact ...orders-api\target\orders-api-1.0.0.jar with repackaged archive, adding nested
dependencies in BOOT-INF/.
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 4.452 s
```

`target\orders-api-1.0.0.jar` is a **fat jar**: your classes plus every library they need, Tomcat and the Kafka client included. You can copy that one file to any machine with Java 21 and run it.

Exercise 1.3 · Run

Task: start the app and find the log lines that show it is connected to Kafka.

✓ SOLUTION

```
java -jar target\orders-api-1.0.0.jar
```

POWERSHELL

`.\mvnw.cmd spring-boot:run` works too: it builds and starts the app in one step. Abridged log, columns trimmed:

```

:: Spring Boot ::                (v4.1.1)

06:30:17.564 [main] OrdersApiApplication : Starting OrdersApiApplication v1.0.0 using Java 21.0.12
06:30:21.138 [main] TomcatWebServer      : Tomcat initialized with port 8090 (http)
06:30:24.501 [main] TomcatWebServer      : Tomcat started on port 8090 (http) with context path '/'
06:30:24.969 [main] OrdersApiApplication : Started OrdersApiApplication in 8.886 seconds
06:30:25.300 [-listener-0-C-1] KafkaMessageListenerContainer : orders-api: partitions assigned: [orders-0, orders-1,
orders-2]
```

OUTPUT

The last line is the one that matters. The `@KafkaListener` joined the consumer group `orders-api`. It is the group's only member, so the group coordinator gave it **all three partitions** of `orders`. That consumer runs on its own thread (`...-listener-0-C-1`), separate from the web requests.

Leave the app running. Stop it with **Ctrl+C** when you are done.

THE KAFKA CLIENTS ARE QUIET ON PURPOSE

`application.yml` sets `logging.level.org.apache.kafka: WARN`. At `INFO`, the clients print every config value at startup: about 150 lines for the producer and the same again for each consumer. To see them, start with `java -jar target\orders-api-1.0.0.jar --logging.level.org.apache.kafka=INFO`.

Exercise 1.4 · Is the topic there?

Task: check the `orders` topic from the CLI. If it did not exist before, who created it, and with which settings?

✓ SOLUTION

```
kafka-topics.sh --bootstrap-server kafka1:9092 --describe --topic orders
```

KAFKA1 SHELL

```

Topic: orders   TopicId: Gpz6jYWrS1eRMUNaM036tw PartitionCount: 3   ReplicationFactor: 3   Configs:
min.insync.replicas=2
Topic: orders   Partition: 0   Leader: 1   Replicas: 1,2,3   Isr: 1,2,3   Elr:   LastKnownElr:
Topic: orders   Partition: 1   Leader: 2   Replicas: 2,3,1   Isr: 2,3,1   Elr:   LastKnownElr:
Topic: orders   Partition: 2   Leader: 3   Replicas: 3,1,2   Isr: 3,1,2   Elr:   LastKnownElr:
```

OUTPUT

The cluster has `auto.create.topics.enable=false`, so producing to a missing topic would fail. The app declares the topic it needs as a `NewTopic` bean (Lab 2). At startup, Spring's `KafkaAdmin` creates any declared topic that is missing, here with 3 partitions and replication factor 3, the same as Lab 2 of the CLI lab. `min.insync.replicas=2` is the cluster default from `docker-compose.yml`.

If `orders` already exists, for example from the CLI lab, `KafkaAdmin` leaves it alone. The listener then starts by reading every message already in the topic: a **new** group has no committed offsets, and the app sets `auto-offset-reset: earliest`.

RUNNING THE APP SOMEWHERE ELSE

`localhost:19092` only works on the Windows host that runs Docker. For any other location, override the bootstrap servers when you start the app. For example, for a container on the lab's Docker network:

```
java -jar target\orders-api-1.0.0.jar --spring.kafka.bootstrap-servers=kafka1:9092,kafka2:9092,kafka3:9092
```

POWERSHELL

Any property in `application.yml` can be overridden like this, or with an environment variable such as `SPRING_KAFKA_BOOTSTRAP_SERVERS`.

Lab 2 · The Code, File by File

Goal: read the whole application, about 500 lines, and connect each piece to a Kafka idea from the course. Every listing below is the actual file from the project.

The dependencies

Three Spring Boot starters do all the work. The `spring-boot-starter-parent` POM picks matching versions for everything, so the dependencies need no version numbers:

XML

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webmvc</artifactId>    <!-- REST: Spring MVC + Tomcat + Jackson -->
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-kafka</artifactId>    <!-- KafkaTemplate, @KafkaListener, KafkaAdmin -
->
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId> <!-- @Valid, @NotBlank, @Positive -->
</dependency>
```

SPRING BOOT 4 STARTER NAMES

Spring Boot 4 split its auto-configuration into modules, and two starters were renamed. Kafka support now comes from `spring-boot-starter-kafka`. In Boot 3 you added `spring-kafka` directly, and on Boot 4 that alone gives you the library but **no auto-configuration**. Spring MVC comes from `spring-boot-starter-webmvc`, the new name for `spring-boot-starter-web`. Boot 4 also moves to Jackson 3, whose classes live in the `tools.jackson` package instead of `com.fasterxml.jackson`.

The configuration: application.yml

application.yml

YAML

```
# -----
# orders-api - connects to the 3-broker cluster of the Kafka CLI Lab
# (kafka_cli_lab/docker-compose.yml). Run on Windows, so it uses the
# EXTERNAL listeners that the brokers advertise as localhost:19092/29092/39092.
# -----
server:
  port: 8090                # 8080 is often taken; Kafka UI uses 8081

spring:
  application:
    name: orders-api

kafka:
  # a few brokers - one reachable is enough, the client learns the rest
  bootstrap-servers: localhost:19092,localhost:29092,localhost:39092
  client-id: orders-api

  # the same settings as lab/config/producer.properties
  producer:
    key-serializer: org.apache.kafka.common.serialization.StringSerializer
    value-serializer: org.apache.kafka.common.serialization.StringSerializer
    acks: all          # wait for every in-sync replica
```

```

batch-size: 32KB
compression-type: lz4
properties:
  enable.idempotence: true # no duplicates on retry
  linger.ms: 20 # wait up to 20 ms to fill a batch
  max.block.ms: 10000 # wait at most 10 s for metadata (a broker to talk to)
  request.timeout.ms: 10000 # wait at most 10 s for one broker reply
  delivery.timeout.ms: 25000 # give up on a message after 25 s, retries included

# used by the @KafkaListener and by the on-demand reader
consumer:
  key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
  value-deserializer: org.apache.kafka.common.serialization.StringDeserializer
  auto-offset-reset: earliest # a new group starts at the oldest message

jackson:
  mapper:
    sort-properties-alphabetically: false # keep {"order","item","amount"} order

app:
  topic: orders # the topic from Lab 2 of the CLI lab
  listener-group-id: orders-api # the consumer group of the @KafkaListener
  keep-last: 50 # how many received messages GET /received returns

logging:
  level:
    org.apache.kafka: WARN # the Kafka clients are chatty at INFO

```

Spring Boot reads `spring.kafka.*` and builds three things from it: a `ProducerFactory` behind the `KafkaTemplate`, a `ConsumerFactory` behind the listener and the reader, and a `KafkaAdmin`. The settings are the ones from the CLI lab's property files:

lab/config/*.properties	application.yml	Meaning
<code>acks=all</code>	<code>producer.acks: all</code>	The leader answers once every in-sync replica has the record
<code>enable.idempotence=true</code>	<code>producer.properties.enable.idempotence</code>	Retries cannot create duplicates
<code>linger.ms=20</code> , <code>batch.size=32768</code>	<code>properties.linger.ms</code> , <code>batch-size:</code> <code>32KB</code>	Wait up to 20 ms to fill a batch of up to 32 KB
<code>compression.type=lz4</code>	<code>compression-type: lz4</code>	Compress each batch. The console producer ignored this; the app does not
<code>delivery.timeout.ms=120000</code>	<code>delivery.timeout.ms: 25000</code>	Give up sooner, so a REST caller is not kept waiting for 2 minutes
<code>group.id=orders-reporting</code>	<code>app.listener-group-id: orders-api</code>	Set on the <code>@KafkaListener</code> , so the reader can run without a group
<code>auto.offset.reset=earliest</code>	<code>consumer.auto-offset-reset: earliest</code>	A new group starts at the oldest message
<code>enable.auto.commit=true</code>	(not set)	Spring turns auto-commit off and commits itself - see <code>OrderListener</code>

Spring Boot recognises common settings by name, such as `acks`, `batch-size` and `compression-type`. Any other Kafka setting goes under `properties:`, written exactly as Kafka spells it.

Creating the topic: KafkaTopicConfig

`KafkaTopicConfig.java`

JAVA

```
package com.training.kafka.orders;

import org.apache.kafka.clients.admin.NewTopic;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.kafka.config.TopicBuilder;

@Configuration
public class KafkaTopicConfig {

    /**
     * The orders topic exactly as Lab 2 creates it: 3 partitions, replication factor 3.
     * Spring's KafkaAdmin creates it at startup if it is missing and leaves an existing
     * topic alone - the cluster has auto.create.topics.enable=false.
     */
    @Bean
    NewTopic ordersTopic(@Value("${app.topic}") String topic) {
        return TopicBuilder.name(topic).partitions(3).replicas(3).build();
    }
}
```

`TopicBuilder` is the code version of `kafka-topics.sh --create`. `KafkaAdmin` checks every `NewTopic` bean at startup. It creates missing topics and adds partitions if a bean asks for more than the topic has. It never deletes anything and never reduces partitions.

The data: Order, OrderRequest and friends

Order.java

JAVA

```
package com.training.kafka.orders;

/** The message value - the same JSON as Lab/data/orders.txt: {"order":2001,"item":"keyboard","amount":45} */
public record Order(int order, String item, int amount) {
}
```

OrderRequest.java

JAVA

```
package com.training.kafka.orders;

import jakarta.validation.constraints.NotBlank;
import jakarta.validation.constraints.Positive;

/** The body of POST /api/orders. The customer becomes the message key, the rest the value. */
public record OrderRequest(
    @NotBlank String customer,
    @Positive int order,
    @NotBlank String item,
    @Positive int amount) {

    Order toOrder() {
        return new Order(order, item, amount);
    }
}
```

Java **records** are ideal for messages: they are immutable, and Jackson turns them into JSON and back with no extra code. The REST client sends an `OrderRequest`. The app splits it in two: `customer` becomes the **key**, and the rest, an `Order`, becomes the **value**. The validation annotations reject a bad request before anything reaches Kafka.

`SendReceipt`, `OrderMessage` and `TopicInfo` are records too. They describe what the endpoints return.

Sending: OrderProducer

OrderProducer.java

JAVA

```
package com.training.kafka.orders;

import java.time.Instant;
import java.util.concurrent.ExecutionException;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.TimeoutException;

import org.apache.kafka.clients.producer.RecordMetadata;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.core.NestedExceptionUtils;
import org.springframework.kafka.core.KafkaTemplate;
import org.springframework.kafka.support.SendResult;
import org.springframework.stereotype.Service;
import tools.jackson.databind.json.JsonMapper;

@Service
public class OrderProducer {

    private static final Logger log = LoggerFactory.getLogger(OrderProducer.class);

    private final KafkaTemplate<String, String> kafka;
    private final JsonMapper json;
    private final String topic;

    public OrderProducer(KafkaTemplate<String, String> kafka, JsonMapper json,
        @Value("${app.topic}") String topic) {
        this.kafka = kafka;
        this.json = json;
        this.topic = topic;
    }

    /**
     * Sends one order and waits for the broker's answer, so the caller learns the
     * partition and offset. Key = customer: all of a customer's orders go to the same
     * partition, so they stay in order.
     */
    public SendReceipt send(OrderRequest request) {
        Order order = request.toOrder();
        String value = json.writeValueAsString(order);
        try {
            SendResult<String, String> result =
                kafka.send(topic, request.customer(), value).get(30, TimeUnit.SECONDS);
            RecordMetadata meta = result.getRecordMetadata();
            log.info("sent key={} -> {}-{} @ offset {}", request.customer(), meta.topic(),
                meta.partition(), meta.offset());
            return new SendReceipt(meta.topic(), meta.partition(), meta.offset(),
                Instant.ofEpochMilli(meta.timestamp()), request.customer(), order);
        } catch (ExecutionException e) {
            throw new KafkaUnavailableException("Kafka did not accept the message: "
                + NestedExceptionUtils.getMostSpecificCause(e).getMessage(), e);
        } catch (TimeoutException e) {
            throw new KafkaUnavailableException("No answer from Kafka within 30 seconds", e);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            throw new KafkaUnavailableException("Interrupted while sending", e);
        }
    }
}
```

What happens on each line:

1. `json.writeValueAsString(order)` turns the record into `{"order":2001,"item":"keyboard","amount":45}`. Kafka itself only ever sees bytes. The `StringSerializer` from `application.yml` turns that string into UTF-8 bytes.
2. `kafka.send(topic, key, value)` returns at once with a `CompletableFuture`. The record now waits in the producer's buffer, for up to `linger.ms`, so it can travel in a batch with others.
3. `.get(30, SECONDS)` waits for the future to complete. That means the batch was sent, the leader wrote it, and with `acks=all` every in-sync replica has it too.
4. `RecordMetadata` is the broker's answer: the partition and offset where the record now lives. The endpoint returns it to the caller.

BLOCKING ON GET() - FINE HERE, NOT EVERYWHERE

Waiting for each send keeps the REST API simple: the caller sees the partition and offset straight away. A high-volume producer would not wait. It would attach a callback, `send(...).whenComplete((result, error) -> ...)`, and let batching work. Blocking sends one batch per request, so `linger.ms` buys nothing.

Receiving: OrderListener

OrderListener.java

JAVA

```
package com.training.kafka.orders;

import java.util.Deque;
import java.util.List;
import java.util.concurrent.ConcurrentLinkedDeque;

import org.apache.kafka.clients.consumer.ConsumerRecord;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.kafka.annotation.KafkaListener;
import org.springframework.stereotype.Component;
import tools.jackson.databind.json.JsonMapper;

/**
 * A consumer that runs for as long as the app does: Spring calls poll() in a loop on its
 * own thread and hands every record to onMessage(). It is a member of the consumer group
 * app.listener-group-id, and Spring commits the group's offsets after each batch.
 */
@Component
public class OrderListener {

    private static final Logger log = LoggerFactory.getLogger(OrderListener.class);

    private final Deque<OrderMessage> received = new ConcurrentLinkedDeque<>();
    private final JsonMapper json;
    private final int keepLast;

    public OrderListener(JsonMapper json, @Value("${app.keep-last}") int keepLast) {
        this.json = json;
        this.keepLast = keepLast;
    }

    @KafkaListener(id = "orders-listener", topics = "${app.topic}", groupId = "${app.listener-group-id}")
    public void onMessage(ConsumerRecord<String, String> record) {
        log.info("received key={}<- {}-{} @ offset {}", record.key(), record.topic(),
            record.partition(), record.offset());
        received.addFirst(OrderMessage.from(record, json));
        while (received.size() > keepLast) {
            received.pollLast();
        }
    }
}
```

```

    /** The most recently received messages, newest first. */
    public List<OrderMessage> latest() {
        return List.copyOf(received);
    }
}

```

This is the poll loop from the Consumers lesson, written by Spring. For each `@KafkaListener`, Spring starts a **listener container**: a thread that subscribes to the topic, calls `poll()` forever, and passes each record to your method. What you saw in the Consumers lesson:

- **Group:** `groupId = "orders-api"`. Start a second copy of the app and the two share the partitions (Lab 5).
- **Offsets:** Spring sets `enable.auto.commit=false` and commits the group's offsets itself, after your method has handled every record of a `poll()`. This is the container's default `AckMode.BATCH`. If the app crashes in the middle of a batch, the records of that batch are delivered again: **at-least-once**.
- **Threads:** `onMessage` runs on the container thread, not on a web request thread. The deque is thread-safe because the web threads read it while the listener writes to it.

The deque keeps only the latest 50 messages, in memory. Restart the app and it is empty, but the topic is not.

Reading from any offset: OrderReader

`OrderReader.java`

JAVA

```

package com.training.kafka.orders;

import java.time.Duration;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
import java.util.Map;
import java.util.concurrent.atomic.AtomicInteger;

import org.apache.kafka.clients.consumer.Consumer;
import org.apache.kafka.clients.consumer.ConsumerRecord;
import org.apache.kafka.common.KafkaException;
import org.apache.kafka.common.TopicPartition;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.kafka.core.ConsumerFactory;
import org.springframework.stereotype.Service;
import tools.jackson.databind.json.JsonMapper;

/**
 * Reads the topic on request, from any offset - like kafka-console-consumer.sh with
 * --partition and --offset. It uses assign() and seek() instead of a consumer group, so it
 * commits nothing and does not disturb the listener's group.
 */
@Service
public class OrderReader {

    private static final Duration MAX_WAIT = Duration.ofSeconds(5);

    private final ConsumerFactory<String, String> consumers;
    private final JsonMapper json;
    private final String topic;
    private final AtomicInteger readerNo = new AtomicInteger();

    public OrderReader(ConsumerFactory<String, String> consumers, JsonMapper json,
        @Value("${app.topic}") String topic) {
        this.consumers = consumers;
        this.json = json;
        this.topic = topic;
    }
}

```

```

/**
 * Up to {@code limit} messages from one partition (or all of them), starting at
 * {@code fromOffset} in each, optionally only those whose key is {@code customer}.
 */
public List<OrderMessage> read(Integer partition, long fromOffset, int limit, String customer) {
    try (Consumer<String, String> consumer =
        consumers.createConsumer(null, "orders-reader-",
            String.valueOf(readerNo.incrementAndGet()))) {

        List<TopicPartition> partitions = consumer.partitionsFor(topic).stream()
            .map(p -> new TopicPartition(topic, p.partition()))
            .filter(tp -> partition == null || tp.partition() == partition)
            .toList();
        if (partitions.isEmpty()) {
            throw new IllegalArgumentException("Topic " + topic + " has no partition " + partition);
        }

        consumer.assign(partitions);
        Map<TopicPartition, Long> first = consumer.beginningOffsets(partitions);
        Map<TopicPartition, Long> end = consumer.endOffsets(partitions);
        for (TopicPartition tp : partitions) {
            // stay inside [first, end] - an offset past the end would trigger auto.offset.reset
            consumer.seek(tp, Math.min(Math.max(fromOffset, first.get(tp)), end.get(tp)));
        }

        List<OrderMessage> out = new ArrayList<>();
        long deadline = System.nanoTime() + MAX_WAIT.toNanos();
        while (out.size() < limit && !reachedEnd(consumer, end) && System.nanoTime() < deadline) {
            for (ConsumerRecord<String, String> record : consumer.poll(Duration.ofMillis(500))) {
                if (customer == null || customer.equals(record.key())) {
                    out.add(OrderMessage.from(record, json));
                }
                if (out.size() == limit) {
                    break;
                }
            }
        }

        out.sort(Comparator.comparingInt(OrderMessage::partition).thenComparingLong(OrderMessage::offset));
        return out;
    } catch (KafkaException e) {
        throw new KafkaUnavailableException("Could not read from Kafka: " + e.getMessage(), e);
    }
}

private static boolean reachedEnd(Consumer<String, String> consumer, Map<TopicPartition, Long> end) {
    return end.entrySet().stream().allMatch(e -> consumer.position(e.getKey()) >= e.getValue());
}
}

```

The listener is a group member: it always moves forward and remembers its place. The reader is the opposite. It is

`kafka-console-consumer.sh --partition --offset` in Java:

- `createConsumer(null, ...)` creates a consumer with **no group id**. Without a group it cannot `subscribe()`, so it calls `assign()` and picks the partitions itself.
- `beginningOffsets()` and `endOffsets()` give the first offset still on disk and the next one to be written.
- `seek()` jumps to the requested offset. It is clamped to that range, because a position past the end makes the consumer fall back to `auto.offset.reset`.
- The loop polls until it has `limit` messages, has reached the end of every partition, or has used up its 5 seconds.

It commits nothing and joins no group, so calling it never moves the listener's position.

Describing the topic: TopicInspector

TopicInspector.java

JAVA

```
package com.training.kafka.orders;

import java.util.List;
import java.util.Map;
import java.util.concurrent.ExecutionException;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.TimeoutException;
import java.util.function.Function;
import java.util.stream.Collectors;

import org.apache.kafka.clients.admin.Admin;
import org.apache.kafka.clients.admin.ListOffsetsResult.ListOffsetsResultInfo;
import org.apache.kafka.clients.admin.OffsetSpec;
import org.apache.kafka.clients.admin.TopicDescription;
import org.apache.kafka.common.Node;
import org.apache.kafka.common.TopicPartition;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.core.NestedExceptionUtils;
import org.springframework.kafka.core.KafkaAdmin;
import org.springframework.stereotype.Service;

/** The Admin API view of the topic: leaders, replicas, ISR and offsets per partition. */
@Service
public class TopicInspector {

    private final KafkaAdmin kafkaAdmin;
    private final String topic;

    public TopicInspector(KafkaAdmin kafkaAdmin, @Value("${app.topic}") String topic) {
        this.kafkaAdmin = kafkaAdmin;
        this.topic = topic;
    }

    public TopicInfo describe() {
        try (Admin admin = Admin.create(kafkaAdmin.getConfigurationProperties())) {
            TopicDescription description = admin.describeTopics(List.of(topic))
                .allTopicNames().get(10, TimeUnit.SECONDS).get(topic);
            List<TopicPartition> partitions = description.partitions().stream()
                .map(p -> new TopicPartition(topic, p.partition()))
                .toList();
            Map<TopicPartition, ListOffsetsResultInfo> first = offsets(admin, partitions,
                OffsetSpec.earliest());
            Map<TopicPartition, ListOffsetsResultInfo> end = offsets(admin, partitions, OffsetSpec.latest());

            return new TopicInfo(topic, description.partitions().stream().map(p -> {
                TopicPartition tp = new TopicPartition(topic, p.partition());
                return new TopicInfo.Partition(p.partition(), p.leader() == null ? -1 : p.leader().id(),
                    ids(p.replicas()), ids(p.isr()), first.get(tp).offset(), end.get(tp).offset());
            }).toList());
        } catch (ExecutionException | TimeoutException e) {
            throw new KafkaUnavailableException("Could not describe topic " + topic + ": "
                + NestedExceptionUtils.getMostSpecificCause(e).getMessage(), e);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            throw new KafkaUnavailableException("Interrupted while describing " + topic, e);
        }
    }

    private static Map<TopicPartition, ListOffsetsResultInfo> offsets(Admin admin, List<TopicPartition>
        partitions,
        OffsetSpec spec) throws ExecutionException, InterruptedException, TimeoutException {
        Map<TopicPartition, OffsetSpec> request = partitions.stream()
            .collect(Collectors.toMap(Function.identity(), tp -> spec));
        return admin.listOffsets(request).all().get(10, TimeUnit.SECONDS);
    }
}
```

```
private static List<Integer> ids(List<Node> nodes) {
    return nodes.stream().map(Node::id).toList();
}
}
```

The **Admin API** is what `kafka-topics.sh` and `kafka-consumer-groups.sh` use internally. `describeTopics()` returns the leader, replicas and ISR of each partition, and `listOffsets()` returns the first and next offset.

`KafkaAdmin.getConfigurationProperties()` reuses the connection settings from `application.yml`, so the bootstrap servers are not repeated.

The REST layer: OrderController and ApiExceptionHandler

OrderController.java

JAVA

```
package com.training.kafka.orders;

import java.util.List;

import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.ResponseStatus;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api/orders")
public class OrderController {

    private final OrderProducer producer;
    private final OrderListener listener;
    private final OrderReader reader;
    private final TopicInspector inspector;

    public OrderController(OrderProducer producer, OrderListener listener, OrderReader reader,
        TopicInspector inspector) {
        this.producer = producer;
        this.listener = listener;
        this.reader = reader;
        this.inspector = inspector;
    }

    /** Send one order to the topic. Returns the partition and offset Kafka stored it at. */
    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public SendReceipt send(@Valid @RequestBody OrderRequest request) {
        return producer.send(request);
    }

    /** The messages the @KafkaListener has received, newest first. */
    @GetMapping("/received")
    public List<OrderMessage> received() {
        return listener.latest();
    }

    /** Read the topic itself, from any offset - nothing is removed by reading. */
    @GetMapping
    public List<OrderMessage> read(@RequestParam(required = false) Integer partition,
        @RequestParam(defaultValue = "0") long offset,
        @RequestParam(defaultValue = "20") int limit,
        @RequestParam(required = false) String customer) {
```

```
        return reader.read(partition, offset, Math.clamp(limit, 1, 500), customer);
    }

    /** Partitions, Leaders, replicas, ISR and offsets - like kafka-topics.sh --describe. */
    @GetMapping("/topic")
    public TopicInfo topic() {
        return inspector.describe();
    }
}
```

The controller only translates between HTTP and the four services: no Kafka code lives here. `@Valid` triggers the checks on `OrderRequest`, and `Math.clamp` keeps `limit` between 1 and 500.

`ApiExceptionHandler` turns failures into **problem JSON** (RFC 9457): a 400 for a bad request, with the invalid fields listed, and a 503 when Kafka cannot be reached. You will see both in Lab 3.

Lab 3 · Send Messages over REST

Goal: send orders through `POST /api/orders` with `curl.exe`, PowerShell and a script, then confirm with the CLI that they are in the topic, in the partitions their keys chose.

The app must be running (Lab 1). Run the commands in the `orders-api` folder.

Exercise 3.1 · Your first order, with curl.exe

Task: send alice's order 2001 from `samples\order.json` and read the response. Which partition and offset did Kafka give it?

✓ SOLUTION

POWERSHELL

```
type samples\order.json
curl.exe -i -X POST http://localhost:8090/api/orders -H "Content-Type: application/json" --data
"@samples/order.json"
```

OUTPUT

```
{"customer": "alice", "order": 2001, "item": "keyboard", "amount": 45}

HTTP/1.1 201
Content-Type: application/json

{"topic":"orders","partition":0,"offset":0,"timestamp":"2026-10-01T01:00:31.397Z","key":"alice","value":
{"order":2001,"item":"keyboard","amount":45}}
```

201 Created, and the body is the receipt: partition **0**, offset **0**, the first message ever written to that partition. The `timestamp` was set by the producer when it sent the record. It is UTC, which is why it reads 01:00 for a send made at 06:30 IST.

TYPE CURL.EXE, NOT CURL

In Windows PowerShell, `curl` is an **alias for** `Invoke-WebRequest`, which takes different options, so `curl -X POST ...` fails with a confusing error. `curl.exe` runs the real curl that ships with Windows.

Exercise 3.2 · JSON on the command line – the PowerShell trap

Task: pass the JSON directly with `-d` instead of from a file. Try it with plain single quotes first.

✓ SOLUTION

To keep the topic clean, these tries send an **invalid** order, so even a request that gets through is rejected before it reaches Kafka:

POWERSHELL

```
curl.exe -s -X POST http://localhost:8090/api/orders -H "Content-Type: application/json" -d
'{"customer":"","order":0,"item":"pen","amount":-5}'
curl.exe -s -X POST http://localhost:8090/api/orders -H "Content-Type: application/json" -d
'{"customer":"","order":0,"item":"pen","amount":-5}'
```

OUTPUT

```
{"title":"Bad Request","status":400,"detail":"Failed to read request","instance":"/api/orders"}
{"title":"Bad Request","status":400,"detail":"The order is not valid - nothing was sent to"}
```

```
Kafka", "instance": "/api/orders", "errors": { "amount": "must be greater than 0", "customer": "must not be blank", "order": "must be greater than 0" }}
```

The first request never arrived as JSON. **Windows PowerShell 5.1 removes the inner double quotes** when it passes an argument to a program such as `curl.exe`, so the server received `{customer:,order:0,...}` and could not parse it: *Failed to read request*. With the quotes escaped as `\`, the JSON arrives intact and the server's validation answers instead.

Three ways to avoid the trap: a file with `--data "@file.json"` (Exercise 3.1), `Invoke-RestMethod` (next exercise), or PowerShell 7, which passes the quotes through correctly.

Exercise 3.3 · Send with PowerShell

Task: send bob's order 2000 (a pen drive for 12) with `Invoke-RestMethod`.

✓ SOLUTION

POWERSHELL

```
$body = @{ customer = "bob"; order = 2000; item = "pen drive"; amount = 12 } | ConvertTo-Json
Invoke-RestMethod -Method Post -Uri http://localhost:8090/api/orders -ContentType 'application/json' -
Body $body
```

OUTPUT

```
topic      : orders
partition  : 0
offset     : 1
timestamp  : 2026-10-01T01:02:25.827Z
key        : bob
value      : @{order=2000; item=pen drive; amount=12}
```

`Invoke-RestMethod` builds the JSON itself with `ConvertTo-Json`, so there is no quoting to get wrong. It also parses the response into an object you can use: `$r.partition`, `$r.value.item`. Bob also landed in partition 0, at the next offset, 1.

Exercise 3.4 · Load the CLI lab's orders

Task: run `send-orders.ps1` from the lab folder. It posts every line of `kafka_cli_lab\lab\data\orders.txt`. Which partition does each customer get? Do all of one customer's orders land in the same place?

✓ SOLUTION

POWERSHELL

```
cd ..
powershell -ExecutionPolicy Bypass -File .\send-orders.ps1
```

OUTPUT

```
alice order 2001 -> orders-0 @ offset 2
dave order 2002 -> orders-1 @ offset 0
carol order 2003 -> orders-2 @ offset 0
alice order 2004 -> orders-0 @ offset 3
erin order 2005 -> orders-2 @ offset 1
frank order 2006 -> orders-1 @ offset 1
bob order 2007 -> orders-0 @ offset 4
dave order 2008 -> orders-1 @ offset 2
carol order 2009 -> orders-2 @ offset 2
alice order 2010 -> orders-0 @ offset 5
```

Partition	Customers
0	alice, bob
1	dave, frank
2	carol, erin

The key decides the partition: `murmur2(key) % 3`, exactly as with the console producer in the CLI lab. All of alice's orders went to partition 0, at offsets 0, 2, 3 and 5, so they are stored, and will be read, in the order she placed them. Offsets count **per partition**: partition 1 starts again at 0.

`-ExecutionPolicy Bypass` lets this one script run even when Windows blocks unsigned scripts. It changes no setting.

ORDER 2001 IS IN THE TOPIC TWICE

`orders.txt` starts with alice's order 2001, the same one you sent in Exercise 3.1. Kafka stored it again, at offset 2.

Idempotence only protects against duplicates caused by the producer's own retries. Two separate sends of the same data are two messages. Removing duplicates of that kind is the application's job, for example a consumer that remembers which order numbers it has already handled.

Exercise 3.5 · Check with the CLI

Task: read the whole topic with `kafka-console-consumer.sh` and show the partition, offset and key of every message. Is it what the API reported?

✓ SOLUTION

```
kafka-console-consumer.sh --bootstrap-server kafka1:9092 --topic orders --from-beginning --max-messages 12 \
--formatter-property print.key=true --formatter-property print.partition=true --formatter-property print.offset=true
```

OUTPUT

```
Partition:0 Offset:0    alice  {"order":"2001","item":"keyboard","amount":45}
Partition:0 Offset:1    bob    {"order":"2000","item":"pen drive","amount":12}
Partition:0 Offset:2    alice  {"order":"2001","item":"keyboard","amount":45}
Partition:0 Offset:3    alice  {"order":"2004","item":"headset","amount":60}
Partition:0 Offset:4    bob    {"order":"2007","item":"cable","amount":9}
Partition:0 Offset:5    alice  {"order":"2010","item":"laptop stand","amount":55}
Partition:1 Offset:0    dave   {"order":"2002","item":"monitor","amount":210}
Partition:1 Offset:1    frank  {"order":"2006","item":"dock","amount":140}
Partition:1 Offset:2    dave   {"order":"2008","item":"chair","amount":320}
Partition:2 Offset:0    carol  {"order":"2003","item":"mouse","amount":18}
Partition:2 Offset:1    erin   {"order":"2005","item":"webcam","amount":75}
Partition:2 Offset:2    carol  {"order":"2009","item":"desk lamp","amount":35}
Processed a total of 12 messages
```

Every message is where the API said it would be. To Kafka, the app is just another producer: the CLI consumer reads its messages like any others. The values are plain JSON text, because the app sent them through a

`StringSerializer`.

Exercise 3.6 · A bad order

Task: send an order with an empty customer, order number 0 and a negative amount, and look at the status code and the body.

✓ SOLUTION

POWERSHELL

```
curl.exe -i -X POST http://localhost:8090/api/orders -H "Content-Type: application/json" -d '{\"customer\": \"\", \"order\": 0, \"item\": \"pen\", \"amount\": -5}'
```

OUTPUT

```
HTTP/1.1 400
Content-Type: application/problem+json

{
  "title": "Bad Request",
  "status": 400,
  "detail": "The order is not valid - nothing was sent to Kafka",
  "instance": "/api/orders",
  "errors": {
    "amount": "must be greater than 0",
    "customer": "must not be blank",
    "order": "must be greater than 0"
  }
}
```

(Body formatted for reading.) `@Valid` checked the `OrderRequest` before the controller method ran, so `OrderProducer` was never called. Check with the CLI consumer: the topic still has 12 messages. Validate at the edge. A message in Kafka cannot be taken back, and every consumer will have to deal with it.

Lab 4 · Get Messages over REST

Goal: get messages back out in two ways. The first is the **listener**, a consumer group that keeps up with the topic and remembers its place. The second is the **reader**, which reads any part of the log on demand. You will also watch the group catch up after a restart.

Exercise 4.1 · Messages from outside, while the app is down

Task: stop the app with **Ctrl+C**. With the CLI, write two messages: one for judy as JSON, and one for grace as plain text. Then describe the group `orders-api`.

✓ SOLUTION

```
kafka-console-producer.sh --bootstrap-server kafka1:9092 --topic orders \
  --reader-property parse.key=true --reader-property key.separator=:
>judy:{"order":2016,"item":"usb hub","amount":22}
>grace:order 2017 - typed by hand, not JSON
>^C
kafka-consumer-groups.sh --bootstrap-server kafka1:9092 --describe --group orders-api
```

KAFKA1 SHELL

Consumer group 'orders-api' has no active members.

OUTPUT

GROUP	TOPIC	PARTITION	CURRENT-OFFSET	LOG-END-OFFSET	LAG	CONSUMER-ID
orders-api	orders	0	6	7	1	-
orders-api	orders	1	3	3	0	-
orders-api	orders	2	3	4	1	-

(Spacing tightened. The `HOST` and `CLIENT-ID` columns are left out here and in the rest of the book.) The group still exists while no app is running. Its **committed offsets** are stored in `__consumer_offsets`. Before it stopped, the listener had handled everything: `CURRENT-OFFSET` 6, 3 and 3 are the next offsets it will read. grace went to partition 0 and judy to partition 2, so each of those partitions now has a **lag of 1**: one message the group has not processed yet.

CTRL+C, NOT CLOSING THE WINDOW

Ctrl+C lets Spring shut down cleanly. The listener commits its offsets and **leaves the group**, so the group shows *no active members* at once. If the process is killed instead, the broker only notices when the member's session times out after `session.timeout.ms`, 45 s by default. Until then the dead member still owns its partitions.

Exercise 4.2 · Restart and catch up

Task: start the app again. What does it receive first, and what does the group look like afterwards?

✓ SOLUTION

```
java -jar target\orders-api-1.0.0.jar
```

POWERSHELL

```
06:34:12.301 [main] OrdersApiApplication : Started OrdersApiApplication in 10.011 seconds
06:34:12.526 [-listener-0-C-1] KafkaMessageListenerContainer : orders-api: partitions assigned: [orders-0, orders-1, orders-2]
```

OUTPUT

```
06:34:12.720 [-listener-0-C-1] OrderListener : received key=grace <- orders-0 @ offset 6
06:34:12.801 [-listener-0-C-1] OrderListener : received key=judy <- orders-2 @ offset 3
```

KAFKA1 SHELL

```
kafka-consumer-groups.sh --bootstrap-server kafka1:9092 --describe --group orders-api
```

OUTPUT

GROUP	TOPIC	PARTITION	CURRENT-OFFSET	LOG-END-OFFSET	LAG	CONSUMER-ID
orders-api	orders	0	7	7	0	orders-api-0-d8ba3877-0fbc-4846-b567-c821c4d54034
orders-api	orders	1	3	3	0	orders-api-0-d8ba3877-0fbc-4846-b567-c821c4d54034
orders-api	orders	2	4	4	0	orders-api-0-d8ba3877-0fbc-4846-b567-c821c4d54034

The listener started from the **committed offsets**, 6 and 3, not from the beginning and not from the end. It received exactly the two messages it had missed, and nothing twice. The lag is back to 0. This is the "consumer goes away and comes back" story from the Consumers lesson. `auto-offset-reset: earliest` played no part: it only applies to a group with **no** committed offset.

The `HOST` column, left out above, shows `/172.21.0.1`: Docker's gateway. From inside the Docker network, the app on Windows appears to come from there.

Exercise 4.3 · What did the listener receive?

Task: send heidi's order 2018 (a stylus for 30), then call `GET /api/orders/received`. Look at the app's log too.

✓ SOLUTION

POWERSHELL

```
$body = @{ customer = "heidi"; order = 2018; item = "stylus"; amount = 30 } | ConvertTo-Json
Invoke-RestMethod -Method Post -Uri http://localhost:8090/api/orders -ContentType 'application/json' -
Body $body
curl.exe -s http://localhost:8090/api/orders/received
```

OUTPUT

```
[
  {"topic": "orders", "partition": 1, "offset": 3, "timestamp": "2026-10-01T01:04:34.444Z", "key": "heidi", "value":
{"order": 2018, "item": "stylus", "amount": 30}},
  {"topic": "orders", "partition": 2, "offset": 3, "timestamp": "2026-10-01T01:03:27.361Z", "key": "judy", "value":
{"order": 2016, "item": "usb hub", "amount": 22}},
  {"topic": "orders", "partition": 0, "offset": 6, "timestamp": "2026-10-01T01:03:27.383Z", "key": "grace", "value":
"order 2017 - typed by hand, not JSON"}
]
```

(The POST's receipt is not shown. The list is formatted one message per line.) Three things to notice:

- **Newest first**, and only messages received **since the restart**. The buffer lives in memory, while the topic still holds all 15 messages.
- grace's value came back as a **string**, because it was not JSON. Anyone with access can write anything to a topic. A consumer has to cope with that, or a **schema** has to prevent it (Module 8, Schema Registry).
- judy's and grace's timestamps are from when the **CLI** sent them, not from when the app read them.

The log shows both halves of the round trip:

OUTPUT

```
06:34:34.561 [-listener-0-C-1] OrderListener : received key=heidi <- orders-1 @ offset 3
06:34:34.577 [nio-8090-exec-1] OrderProducer : sent      key=heidi -> orders-1 @ offset 3
```

Received is logged **before** *sent*. The listener thread got the message from Kafka before the web thread had finished logging the broker's acknowledgement. From producer, through three replicas, to consumer took a few milliseconds.

Exercise 4.4 · Read the log from an offset

Task: read three messages from partition 0, starting at offset 2.

✓ SOLUTION

```
curl.exe -s "http://localhost:8090/api/orders?partition=0&offset=2&limit=3"
```

POWERSHELL

OUTPUT

```
[
  {"topic": "orders", "partition": 0, "offset": 2, "timestamp": "2026-10-01T01:02:26.712Z", "key": "alice", "value":
{"order": 2001, "item": "keyboard", "amount": 45}},
  {"topic": "orders", "partition": 0, "offset": 3, "timestamp": "2026-10-01T01:02:27.004Z", "key": "alice", "value":
{"order": 2004, "item": "headset", "amount": 60}},
  {"topic": "orders", "partition": 0, "offset": 4, "timestamp": "2026-10-01T01:02:27.142Z", "key": "bob", "value":
{"order": 2007, "item": "cable", "amount": 9}}
]
```

The CLI equivalent is `kafka-console-consumer.sh --partition 0 --offset 2 --max-messages 3`. Put the URL in quotes in PowerShell: an unquoted `&` is an operator.

Exercise 4.5 · One customer's orders

Task: get all of alice's orders. How many partitions did the reader have to look in?

✓ SOLUTION

```
curl.exe -s "http://localhost:8090/api/orders?customer=alice"
```

POWERSHELL

OUTPUT

```
[
  {"topic": "orders", "partition": 0, "offset": 0, "timestamp": "2026-10-01T01:00:31.397Z", "key": "alice", "value":
{"order": 2001, "item": "keyboard", "amount": 45}},
  {"topic": "orders", "partition": 0, "offset": 2, "timestamp": "2026-10-01T01:02:26.712Z", "key": "alice", "value":
{"order": 2001, "item": "keyboard", "amount": 45}},
  {"topic": "orders", "partition": 0, "offset": 3, "timestamp": "2026-10-01T01:02:27.004Z", "key": "alice", "value":
{"order": 2004, "item": "headset", "amount": 60}},
  {"topic": "orders", "partition": 0, "offset": 5, "timestamp": "2026-10-01T01:02:27.249Z", "key": "alice", "value":
{"order": 2010, "item": "laptop stand", "amount": 55}}
]
```

All four are in partition 0, in the order they were sent, with the duplicate 2001 included. The reader still scanned **all three** partitions: Kafka has no index by key, so it filtered the records in the app. Looking up by key is a job for a database or a Kafka Streams state store, which a topic is not.

Exercise 4.6 · Reading does not delete

Task: read the whole topic twice. Does the second read return the same messages? Did the listener's group move?

✓ SOLUTION

POWERSHELL

```
(Invoke-RestMethod "http://localhost:8090/api/orders?limit=50").Count
```

OUTPUT

15

That is 7 + 4 + 4 messages in the three partitions. Run it again as often as you like: the answer stays 15 until someone writes more. A topic is a **log, not a queue**, and consuming removes nothing. The reader uses `assign()` with no group and commits nothing, so `kafka-consumer-groups.sh --describe --group orders-api` shows the same offsets as before. There is also no new group in `--list`.

Exercise 4.7 · Describe the topic over REST

Task: call `GET /api/orders/topic` and compare it with `kafka-topics.sh --describe` from Exercise 1.4.

✓ SOLUTION

POWERSHELL

```
curl.exe -s http://localhost:8090/api/orders/topic
```

OUTPUT

```
{
  "topic": "orders",
  "partitions": [
    {
      "partition": 0,
      "leader": 1,
      "replicas": [1, 2, 3],
      "isr": [1, 2, 3],
      "startOffset": 0,
      "endOffset": 7
    },
    {
      "partition": 1,
      "leader": 2,
      "replicas": [2, 3, 1],
      "isr": [2, 3, 1],
      "startOffset": 0,
      "endOffset": 4
    },
    {
      "partition": 2,
      "leader": 3,
      "replicas": [3, 1, 2],
      "isr": [3, 1, 2],
      "startOffset": 0,
      "endOffset": 4
    }
  ]
}
```

Same leaders, replicas and ISR as the CLI shows, because the CLI uses the same Admin API. `endOffset` is the **next** offset to be written. `endOffset - startOffset` is the number of messages still in the partition, as long as retention has not deleted any and the topic is not compacted.

ERRORS ARE JSON TOO

Ask for a partition that does not exist:

POWERSHELL

```
curl.exe -s "http://localhost:8090/api/orders?partition=7"
```

OUTPUT

```
{
  "title": "Bad Request",
  "status": 400,
  "detail": "Topic orders has no partition 7",
  "instance": "/api/orders"
}
```

Lab 5 · Failures and Scaling

Goal: see the course's ideas work in a real application. Stop a broker, and the app keeps sending. Start a second copy of the app, and the consumer group splits the partitions between the two.

Exercise 5.1 · A broker fails

Task: stop `kafka2`, then send two orders: ivan's 2019 and dave's 2020. Do they succeed, and how long do they take? Then describe the topic.

✓ SOLUTION

POWERSHELL

```
docker stop kafka2
curl.exe -s -w "\nHTTP %{http_code} in %{time_total}s\n" -X POST http://localhost:8090/api/orders -H
"Content-Type: application/json" -d '{"customer":"ivan","order":2019,"item":"hdmi
cable","amount":15}'
curl.exe -s -w "\nHTTP %{http_code} in %{time_total}s\n" -X POST http://localhost:8090/api/orders -H
"Content-Type: application/json" -d
'{"customer":"dave","order":2020,"item":"footrest","amount":40}'
```

OUTPUT

```
kafka2
{"topic":"orders","partition":2,"offset":4,"timestamp":"2026-10-01T01:04:55.984Z","key":"ivan","value":
{"order":2019,"item":"hdmi cable","amount":15}}
HTTP 201 in 0.052261s
{"topic":"orders","partition":1,"offset":4,"timestamp":"2026-10-01T01:04:56.152Z","key":"dave","value":
{"order":2020,"item":"footrest","amount":40}}
HTTP 201 in 0.059433s
```

POWERSHELL

```
curl.exe -s http://localhost:8090/api/orders/topic
```

OUTPUT

```
{"topic": "orders", "partitions": [
  {"partition": 0, "leader": 1, "replicas": [1, 2, 3], "isr": [1, 3], "startOffset": 0, "endOffset": 7},
  {"partition": 1, "leader": 3, "replicas": [2, 3, 1], "isr": [3, 1], "startOffset": 0, "endOffset": 5},
  {"partition": 2, "leader": 3, "replicas": [3, 1, 2], "isr": [3, 1], "startOffset": 0, "endOffset": 5}
]}
```

Both sends succeeded in about 50 ms, and the app noticed nothing. Broker 2 had led partition 1. The controller made broker **3** the new leader, and the producer's next metadata refresh pointed it there. Broker 2 also dropped out of every **ISR**. With `acks=all`, a write now waits for two replicas instead of three. That is still allowed, because `min.insync.replicas=2`.

Exercise 5.2 · The broker comes back

Task: start `kafka2` again. When it is healthy, describe the topic once more. Is everything as before?

✓ SOLUTION

POWERSHELL

```
docker start kafka2
docker inspect -f "{{.State.Health.Status}}" kafka2      # repeat until it says healthy
curl.exe -s http://localhost:8090/api/orders/topic
```

OUTPUT

```
{
  "topic": "orders",
  "partitions": [
    {
      "partition": 0,
      "leader": 1,
      "replicas": [1, 2, 3],
      "isr": [1, 2, 3],
      "startOffset": 0,
      "endOffset": 7
    },
    {
      "partition": 1,
      "leader": 3,
      "replicas": [2, 3, 1],
      "isr": [1, 2, 3],
      "startOffset": 0,
      "endOffset": 5
    },
    {
      "partition": 2,
      "leader": 3,
      "replicas": [3, 1, 2],
      "isr": [1, 2, 3],
      "startOffset": 0,
      "endOffset": 5
    }
  ]
}
```

Broker 2 copied the messages it had missed, ivan's and dave's, from the leaders and **rejoined every ISR**. But partition 1 is still led by broker **3**, even though its preferred leader is the first replica in the list, broker 2. The cluster moves leadership back on its own within about 5 minutes (`leader.imbalance.check.interval.seconds=300`). You can also do it at once with `kafka-leader-election.sh --election-type preferred`, as in Lab 3 of the CLI lab.

WHAT IF TWO BROKERS STOP?

Try it yourself and work out the answer first. With only one broker left, the ISR cannot reach `min.insync.replicas=2`, so an `acks=all` write cannot succeed. The KRaft quorum has also lost its majority, 1 vote of 3. The producer keeps retrying until `delivery.timeout.ms` (25 s) runs out. The app then answers **503** with the `Kafka unavailable` problem JSON. Start both brokers again before you go on.

Exercise 5.3 · Scale out: a second instance

Task: leave the first instance running. Start a second one on port 8091, in a second PowerShell window, then look at both logs and at the group.

✓ SOLUTION

POWERSHELL

```
java -jar target\orders-api-1.0.0.jar --server.port=8091
```

First instance (8090):

OUTPUT

```
06:34:12.526 [-listener-0-C-1] KafkaMessageListenerContainer : orders-api: partitions assigned: [orders-0, orders-1, orders-2]
06:36:21.623 [-listener-0-C-1] KafkaMessageListenerContainer : orders-api: partitions revoked: [orders-0, orders-1, orders-2]
06:36:21.636 [-listener-0-C-1] KafkaMessageListenerContainer : orders-api: partitions assigned: [orders-2]
```

Second instance (8091):

OUTPUT

```
06:36:21.658 [-listener-0-C-1] KafkaMessageListenerContainer : orders-api: partitions assigned: [orders-0, orders-1]
```

KAFKA1 SHELL

```
kafka-consumer-groups.sh --bootstrap-server kafka1:9092 --describe --group orders-api
```

OUTPUT

GROUP	TOPIC	PARTITION	CURRENT-OFFSET	LOG-END-OFFSET	LAG	CONSUMER-ID
orders-api	orders	0	7	7	0	orders-api-0-4027d0d1-5f1f-4860-a1c5-586cff2cde43
orders-api	orders	1	5	5	0	orders-api-0-4027d0d1-5f1f-4860-a1c5-586cff2cde43
orders-api	orders	2	5	5	0	orders-api-0-d8ba3877-0fbe-4846-b567-c821c4d54034

That is a **rebalance**. The second instance joined the same group, so the first gave up all its partitions. The group then shared them out again: partitions 0 and 1 to the new member, partition 2 to the old one. Each partition still has **exactly one** reader in the group, so the order within each partition is kept. The two `CONSUMER-ID`s are the two instances.

No code changed. **Scaling a consumer means starting more copies of it, up to the number of partitions.** A fourth instance on this 3-partition topic would get nothing and wait as a standby.

Exercise 5.4 · Who receives what?

Task: send alice's order 2021 (a mouse pad for 8) **to the first instance, on port 8090**. Which instance's listener receives it?

✓ SOLUTION

POWERSHELL

```
curl.exe -s -X POST http://localhost:8090/api/orders -H "Content-Type: application/json" -d
'{"customer\":\"alice\", \"order\":2021, \"item\":\"mouse pad\", \"amount\":8}'
curl.exe -s http://localhost:8091/api/orders/received
```

OUTPUT

```
{"topic":"orders","partition":0,"offset":7,"timestamp":"2026-10-01T01:06:36.888Z","key":"alice","value":
{"order":2021,"item":"mouse pad","amount":8}}
[
  {"topic": "orders", "partition": 0, "offset": 7, "timestamp": "2026-10-01T01:06:36.888Z", "key": "alice", "value":
{"order": 2021, "item": "mouse pad", "amount": 8}}
]
```

The logs of the two instances:

OUTPUT

```
8090: 06:36:36.919 [nio-8090-exec-7] OrderProducer : sent      key=alice -> orders-0 @ offset 7
8091: 06:36:37.008 [-listener-0-C-1] OrderListener : received key=alice <- orders-0 @ offset 7
```

Instance 8090 **sent** the order, but instance 8091 **received** it, because 8091 owns partition 0. The producer side and the consumer side of an app are independent clients: who sends a message has nothing to do with who reads it. The partition alone decides.

Notice also that 8091's `/received` holds one message, while 8090's holds the older ones. With several instances, an in-memory view is only ever part of the picture. That is one more reason to read from the topic, or from a shared store.

Exercise 5.5 · Scale back in

Task: stop the second instance with Ctrl+C and watch the first instance's log.

✓ SOLUTION

OUTPUT

```
06:39:42.713 [-listener-0-C-1] KafkaMessageListenerContainer : orders-api: partitions revoked: [orders-2]
06:39:42.727 [-listener-0-C-1] KafkaMessageListenerContainer : orders-api: partitions assigned: [orders-0, orders-1,
orders-2]
```

Another rebalance: the survivor takes over all three partitions. It resumes 0 and 1 from the offsets the second instance committed, so nothing is lost and nothing is read twice. With Ctrl+C this happens **at once**, because the leaving instance tells the group coordinator. If an instance crashes, the group waits for `session.timeout.ms`, 45 s, before it moves that instance's partitions. The instance in this capture was killed rather than stopped with Ctrl+C, so the rebalance waited out that timeout.

Challenges

No solutions this time: the earlier exercises give you everything you need.

1. **Replay the whole topic.** Stop the app. Use `kafka-consumer-groups.sh --reset-offsets --to-earliest --group orders-api --topic orders --execute` (CLI Lab 7), then start the app again. What does `/received` show, and why does it hold at most 50 messages?
2. **A second group.** Start an instance with `--app.listener-group-id=orders-audit --server.port=8092`. Which messages does it receive? Does it change anything for the group `orders-api`?
3. **Asynchronous sending.** Change `OrderProducer` so that it does not wait. Return `202 Accepted` at once, and log the partition and offset in a `whenComplete` callback. What does the caller lose?
4. **More partitions.** Change the `NewTopic` bean to 6 partitions and restart. What does `KafkaAdmin` do to the existing topic? Which partitions do alice's **new** orders go to now, and what does that mean for the order of her messages?

Lab 6 · API Reference and Troubleshooting

Endpoints

Base URL: `http://localhost:8090/api/orders`. Every request is also in `orders-api\requests.http`, ready to run from VS Code (*REST Client* extension) or IntelliJ.

POST /api/orders - send an order

Body field	Type	Rule	Becomes
<code>customer</code>	string	not blank	the message key
<code>order</code>	integer	> 0	value field <code>order</code>
<code>item</code>	string	not blank	value field <code>item</code>
<code>amount</code>	integer	> 0	value field <code>amount</code>

Status	When	Body
<code>201 Created</code>	Kafka acknowledged the message (<code>acks=all</code>)	<code>topic</code> , <code>partition</code> , <code>offset</code> , <code>timestamp</code> , <code>key</code> , <code>value</code>
<code>400 Bad Request</code>	Invalid field or malformed JSON	problem JSON. For invalid fields, <code>errors</code> lists them
<code>503 Service Unavailable</code>	No broker answered, or the write failed (e.g. too few in-sync replicas)	problem JSON with the Kafka error in <code>detail</code>

GET /api/orders/received - what the listener received

No parameters. Returns up to `app.keep-last` (50) messages, newest first, received since the app started. Each message has `topic`, `partition`, `offset`, `timestamp`, `key` and `value`. `value` is JSON when the message was JSON, and a string otherwise.

GET /api/orders - read the topic

Parameter	Default	Meaning
<code>partition</code>	all	Read only this partition. Unknown partition: <code>400</code>
<code>offset</code>	<code>0</code>	Start at this offset in each partition read. Clamped to the partition's first and end offset
<code>limit</code>	<code>20</code>	At most this many messages (1-500)
<code>customer</code>	-	Only messages with this key

The result is sorted by partition, then offset. The reader stops at the end of the log, or after 5 seconds. It uses no consumer group and commits nothing.

GET /api/orders/topic - describe the topic

For each partition: `leader`, `replicas`, `isr` (broker ids), `startOffset` (first offset still stored) and `endOffset` (next offset to be written).

Settings

All in `src/main/resources/application.yml`. Override any of them at startup with `--name=value`, or with an environment variable such as `APP_TOPIC`. No rebuild is needed.

Property	Default	Meaning
<code>server.port</code>	<code>8090</code>	HTTP port
<code>spring.kafka.bootstrap-servers</code>	<code>localhost:19092,localhost:29092,localhost:39092</code>	Where to find the cluster
<code>app.topic</code>	<code>orders</code>	The topic the app sends to, listens to and reads
<code>app.listener-group-id</code>	<code>orders-api</code>	The listener's consumer group
<code>app.keep-last</code>	<code>50</code>	Size of the <code>/received</code> buffer
<code>spring.kafka.producer.acks</code>	<code>all</code>	<code>0</code> , <code>1</code> or <code>all</code> - try <code>1</code> with a broker down
<code>logging.level.org.apache.kafka</code>	<code>WARN</code>	<code>INFO</code> shows every client setting at startup

POWERSHELL

```
java -jar target\orders-api-1.0.0.jar --server.port=8095 --app.listener-group-id=orders-test
```

Troubleshooting

Symptom	Cause	Fix
<code>curl -X POST ...</code> fails with <i>A parameter cannot be found that matches parameter name 'X'</i>	<code>curl</code> is PowerShell's alias for <code>Invoke-WebRequest</code>	Type <code>curl.exe</code>
<code>400</code> with <code>"detail": "Failed to read request"</code> from <code>curl.exe -d '{"a":"b"}'</code>	Windows PowerShell 5.1 strips the inner <code>"</code>	Escape them as <code>\</code> , or use <code>--data "@file.json"</code> or <code>Invoke-RestMethod</code> (Lab 3)
Startup fails: <i>Port 8090 was already in use</i>	Another program, or another copy of the app	<code>--server.port=8091</code> , or stop the other one: <code>Get-NetTCPConnection -LocalPort 8090</code> shows its process id
Log repeats <i>Connection to node -1 (localhost/127.0.0.1:19092) could not be established</i>	The cluster is not running	<code>docker compose up -d</code> in <code>kafka_cli_lab</code> , wait for <code>(healthy)</code>
Log shows <i>Connection to node 2 (kafka2/...) could not be established, or UnknownHostException: kafka2</i>	The app reached a broker on the internal listener name	The app must use the EXTERNAL ports <code>19092/29092/39092</code> from Windows, or <code>kafka1:9092 ...</code> from inside the Docker network - never a mix
<code>POST</code> returns <code>503</code> after about 25 s	No in-sync leader can take the write (brokers down, ISR below <code>min.insync.replicas</code>)	<code>docker compose ps</code> , then start the stopped brokers (Lab 5)
<code>/received</code> is empty after a restart	The buffer lives in memory, and the group resumes after its committed offsets	Expected - use <code>GET /api/orders</code> to read older messages

Symptom	Cause	Fix
The listener gets no partitions: log shows <code>partitions assigned: []</code>	More app instances than partitions	Expected - it is a standby. Stop an instance, or add partitions
<code>mvnw.cmd</code> fails: <i>Cannot download ... maven</i>	No internet, or a proxy, on the first build	Set the proxy in <code>%USERPROFILE%\.m2\settings.xml</code> , or install Maven and run <code>mvn -DskipTests package</code>
Build fails with <i>release version 21 not supported</i>	<code>java</code> on the PATH is older than 21	<code>java -version</code> - install JDK 21 and set <code>JAVA_HOME</code>

Where to go next

- **Schema Registry** (Kafka 101, Module 8): replace the hand-made JSON with Avro or JSON Schema, so a producer cannot write something like grace's free-text message.
- **Spring for Apache Kafka reference:** `@RetryableTopic` and dead-letter topics for messages that fail, transactions, and `ReplyingKafkaTemplate` for request/reply.
- **Testing:** Spring Kafka's `@EmbeddedKafka` or Testcontainers' Kafka module start a broker inside a JUnit test.